

Does Feature Regularisation Hold Beyond One Dataset?

Seven datasets, three kinds of problem, every factor held fixed

Experiment report

27 September 2026

Summary. The earlier credit experiment showed two things: a simple linear model on the tree bits matched the best tree model, and removing the deepest tree layers cut overfitting more than tenfold at no cost. This report repeats the same experiment, with every setting held identical, on seven datasets covering two-class, many-class and number-prediction problems. **The regularisation result holds:** on every dataset where the model overfit, deleting the deep layers cut the train-minus-validation gap by 62 to 94 percent, and the training score came down toward the test score, for a test cost never above 0.016. **The “matches the tree” result does not generalise:** credit is the only dataset where the linear model reaches the best tree; elsewhere it covers between 48 and 95 percent of the way from raw features to the tree, and on wine quality almost none. **Deleting beats flipping:** bit-flip noise damages calibration on classification and is destructive on regression, while deletion regularises without side effects.

1 Goal of the experiment

The previous report studied one dataset, credit. With the classifier fixed to a plain logistic regression on the bits of a frozen random forest, two results stood out. First, the linear model matched the best gradient-boosted tree (0.8457 against 0.8453 on the ranking score). Second, regularising the *features* rather than the model, by deleting the deepest tree layers or flipping bits during training, reduced the train-minus-validation gap from 0.0974 to as little as 0.0092 with no change in the test score.

The response to that report set five requirements, and this experiment is built to meet each of them.

1. **Try it on a few more datasets.** Six new datasets were added to credit, chosen to cover two-class problems, many-class problems and number-prediction (regression) problems.
2. **Keep all experimental factors intact.** The forest, the honesty protocol, the head and its penalty, the noise recipe, the row cap, the random seed and the set of arms are identical on every dataset. Nothing was tuned per dataset.
3. **Track all metrics across train, validation and test.** Every run reports three scores on all three splits.
4. **Report that the training accuracy fell from 1.** The tables show training and test side by side for the full encoding and for the shallow one, so the drop is visible directly.
5. **See whether the observation holds.** Each dataset gets an explicit yes-or-no verdict against a rule fixed before the runs.

The question, in one line: *is “regularise the features, not the model” a property of credit, or a property of tree encodings in general?*

2 Datasets

Seven datasets, all from public benchmark collections and all without the identifier-column leakage that spoiled an earlier dataset in this project. Each was split into 70 percent training, 15 percent validation and 15 percent test, with the same random seed, and capped at 16000 rows.

Dataset	What the model has to predict
credit	Whether a person will fall seriously behind on loan payments within two years, from ten numbers describing their finances. Two classes, balanced. The anchor dataset from the previous report.
house_16H	Whether a house is worth more than the typical house in its area, from sixteen census-style numbers. Two classes.
MagicTelescope	Whether a flash recorded by a gamma-ray telescope is a real signal or background noise, from ten shape measurements of the flash. Two classes.
pendigits	Which digit from 0 to 9 was written, from sixteen numbers describing the pen's path. Ten classes, balanced.
covertype	Which of seven forest cover types grows on a patch of land, from fifty-four map-derived measurements. Seven classes, unbalanced. The full dataset has 581,012 rows; a random 16,000 were used.
cpu_act	The fraction of time a computer's processor spends running user programs, from twenty-one activity counters. A number to predict.
wine_quality	The quality score (3 to 9) that a taster gave a wine, from eleven chemical measurements. A number to predict, and known to be difficult.

Dataset	Kind	Raw features	Train	Val	Test	Tree bits	Best tree
credit	two-class	10	11,200	2,400	2,400	5,401	lightgbm 0.8453
house_16H	two-class	16	9,441	2,023	2,024	5,756	lightgbm 0.9526
MagicTelescope	two-class	10	9,363	2,006	2,007	4,684	xgboost 0.9377
pendigits	ten-class	16	7,694	1,649	1,649	5,517	lightgbm 0.9995
covertype	seven-class	54	11,200	2,400	2,400	4,339	xgboost 0.9654
cpu_act	regression	21	5,734	1,229	1,229	3,507	lightgbm 0.9867
wine_quality	regression	11	4,547	975	975	5,498	random forest 0.4722

Table 1: Sizes after the split, the width of the tree encoding each forest produced, and the best of the three reference tree models fitted on the raw features of the same split. “Kind” names the type of problem.

How the scores are read. Three kinds of problem need slightly different scores, and all are reported on all three splits.

- **Ranking score** (classification). How well the model puts the positive cases above the negative ones when sorted by its predicted probability. A value of 0.5 is a coin flip and 1.0 is perfect. For many-class problems it is computed one class against the rest and averaged. This is the primary score for classification.
- **Accuracy** (classification). The fraction of rows the model labels correctly. **Log loss** (classification). A penalty for being confidently wrong; lower is better. It measures how well calibrated the predicted probabilities are, which the ranking score ignores.
- **Explained-variance score** (regression). How much of the spread in the target the predictions account for. 1.0 is perfect, 0 means no better than always guessing the average, and a negative value means worse than that. This is the primary score for regression. Alongside it, the **typical error** (the root of the mean squared error) and the **average absolute error**, both in the target's own units.

Reference tree models. Three tree models were fitted on the raw features of each split with default settings: gradient-boosted trees from two popular libraries (XGBoost and LightGBM) and a random forest. The best of the three on the test split is called “the best tree” throughout and marks the target every other number is compared to.

3 Training architecture and methodology

Every model in this report has two stages. The first is fitted once and then frozen. The second is the only thing that learns.

Stage 1: the tree encoding. A random forest of 100 trees is fitted on the training split, with each tree allowed to grow six levels deep and each leaf required to hold at least five rows. For regression problems the forest is a regression forest; otherwise a classification forest. Every decision node inside every tree becomes one yes-or-no feature, which we call a *bit*: it is 1 if the row goes left at that node and 0 otherwise. Across the whole forest this gives between 3507 and 5756 bits per row, depending on the dataset. The forest is never touched again; it is a way of describing rows, not a predictor.

Depth means reliability. A node near the top of a tree was chosen by looking at thousands of rows. A node at the bottom was chosen by looking at a few dozen. On credit, splits at depths 0 to 2 were fitted on a median of 2,355 rows each, while the deeper splits were fitted on a median of 208. The deep bits are therefore the memorised part of the forest, and roughly half of all bits sit in the single deepest layer. This is why the interventions below are organised by depth.

Honest bits. The forest was fitted using the training labels, so a training row's bits from trees whose random sample *contained* that row were partly carved to classify that very row. Test rows have no such trees. To remove this asymmetry, each training row keeps only the bits from trees that never saw it, on average 36.8 of the 100, rescaled so that magnitudes match. Validation and test rows use all trees. This protocol is identical on every dataset and arm.

Stage 2: the head. A convex linear model with one weight per bit, no hidden layer and no nonlinearity, solved exactly. For classification it is a logistic regression, extended in the usual way to many classes. For regression it is a ridge regression, with the penalty set so the weight on the penalty term matches the classification setting. There is no learning rate, no number of training rounds and no random initialisation anywhere, so no result can be blamed on the optimiser.

The penalty is fixed. The penalty setting is $C = 0.01$ everywhere. A small C means a strong pull of the weights toward zero. It was deliberately not tuned per dataset or per arm, because that would leave any difference in the tables attributable either to the features or to a differently tuned penalty. Holding it fixed makes every difference attributable to the features alone.

How noise is applied. A convex solver sees one fixed table of numbers, so it cannot draw fresh random flips at every training step the way a neural network trainer can. The equivalent is to build three independently flipped copies of the training rows and fit on all of them together, which approximates minimising the loss averaged over the noise. Because stacking three copies triples the data term of the objective, the penalty is rescaled by three so that the pull per original training row is the same as in the arms without noise. Flips are applied to the plain 0/1 bits before the honesty mask. **Validation and test always see clean, unflipped bits.**

How deletion is applied. Deleting a depth means dropping every bit at that depth as a column, from training, validation and test alike. A deleted bit can never be flipped back in. Deletion is a permanent change to the input, so it needs no noisy copies and no rescaling.

How overfitting is measured. The gap is the training score minus the validation score on the primary metric, with the training score computed on *clean* training bits rather than on the flipped table the model was fitted to. Measured the other way, a noisy arm looks like it overfits more purely because its training inputs were corrupted, and the gap stops being comparable between arms. A gap near zero, or negative, means no measurable overfitting.

4 Experimental setup

The eight arms, identical on every dataset. In all of them d is the depth of a split and the deepest layer is depth 5.

Arm	What changes	Why it is there
all bits, no change	nothing	the baseline; also runs the model on raw features alone, as a floor
flip every bit, $p = 0.20$	every bit flipped with probability 0.20	the noise level that worked best on credit
flips rising with depth, $P = 0.2$	flip probability $0.2 \times d/5$; roots never flip	is it better to disturb the unreliable deep bits more?
flip every bit, $p = 0.16$	every bit flipped with probability 0.16	the same <i>total</i> amount of noise as the ramp, spread flat; the fair comparison
delete depth 5	the deepest layer removed	the gentlest deletion
delete depths 4 and 5	the two deepest layers removed	a middle deletion
delete 4 and 5, then ramp 0.2	deletion plus a ramp over the survivors	the combined design
keep only depths 0, 1, 2	everything below depth 2 removed	the configuration recommended from credit

The matched control. For the ramp, the average flip rate over all bits works out to about $0.8 \times P$, so the flat arm at $p = 0.16$ tells the same total number of lies as the ramp at $P = 0.2$. If the ramp beats it, the *shape* of the noise helped. If they tie, only the *amount* mattered.

Bits only. Every arm uses the tree bits alone. If the raw features were included, the model would have an unperturbed shortcut to fall back on, and that would hide what the noise does. The raw-features-only run in the baseline shows how much the bits add.

The rule for “holds”, fixed in advance. The credit observation holds on a dataset if the keep-depths-0-to-2 arm is within 0.01 of the baseline on the test score **and** has a smaller train-minus-validation gap.

Scale. Seven datasets, eight arms, 56 runs, 83 minutes in total on an ordinary processor with no graphics card. Every run’s complete output is kept as its own log.

5 Results

5.1 The verdict, one row per dataset

Dataset	Kind	Best tree	Raw only	All bits			Keep depths 0–2			Test change	Holds
				train	test	gap	train	test	gap		
credit	two-class	0.8453	0.7288	0.9466	0.8457	0.0974	0.8737	0.8459	0.0241	+0.0003	yes
house_16H	two-class	0.9526	0.8850	0.9941	0.9441	0.0471	0.9425	0.9369	0.0034	-0.0071	yes
MagicTelescope	two-class	0.9377	0.8486	0.9678	0.9047	0.0627	0.9041	0.8994	0.0037	-0.0053	yes
pendigits	ten-class	0.9995	0.9906	1.0000	0.9986	0.0015	0.9993	0.9971	0.0023	-0.0015	no
covertypes	seven-class	0.9654	0.9106	0.9890	0.9370	0.0497	0.9505	0.9290	0.0191	-0.0081	yes
cpu_act	regression	0.9867	0.7638	0.9912	0.9758	0.0092	0.9634	0.9599	-0.0015	-0.0159	no
wine_quality	regression	0.4722	0.2800	0.9199	0.2854	0.6045	0.4096	0.2793	0.1180	-0.0061	yes

Table 2: Training and test scores on the primary metric (ranking score for classification, explained-variance score for regression), for the full encoding and for the shallow encoding. “Raw only” is the same head on the raw features with no tree bits. “Test change” is the shallow arm minus the baseline. The observation holds on 5 of 7 datasets.

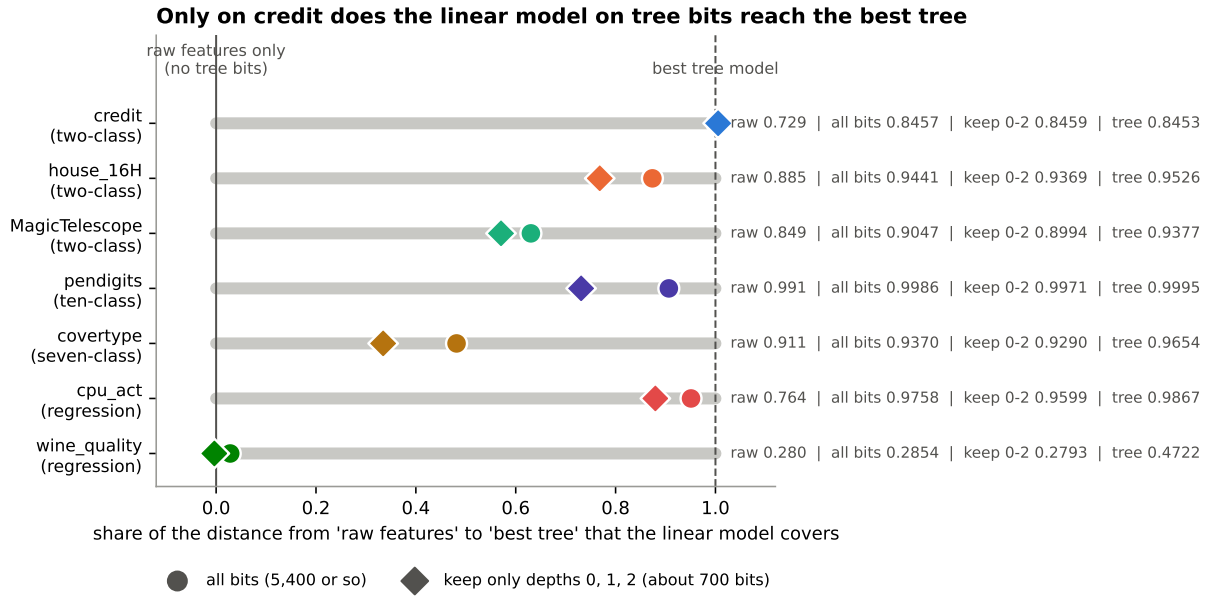


Figure 1: Where the linear model lands between “raw features only” and “best tree” on each dataset. A circle at 1.0 means it reached the tree. Credit is the only dataset where it does; on wine quality the bits add almost nothing a linear model can use.

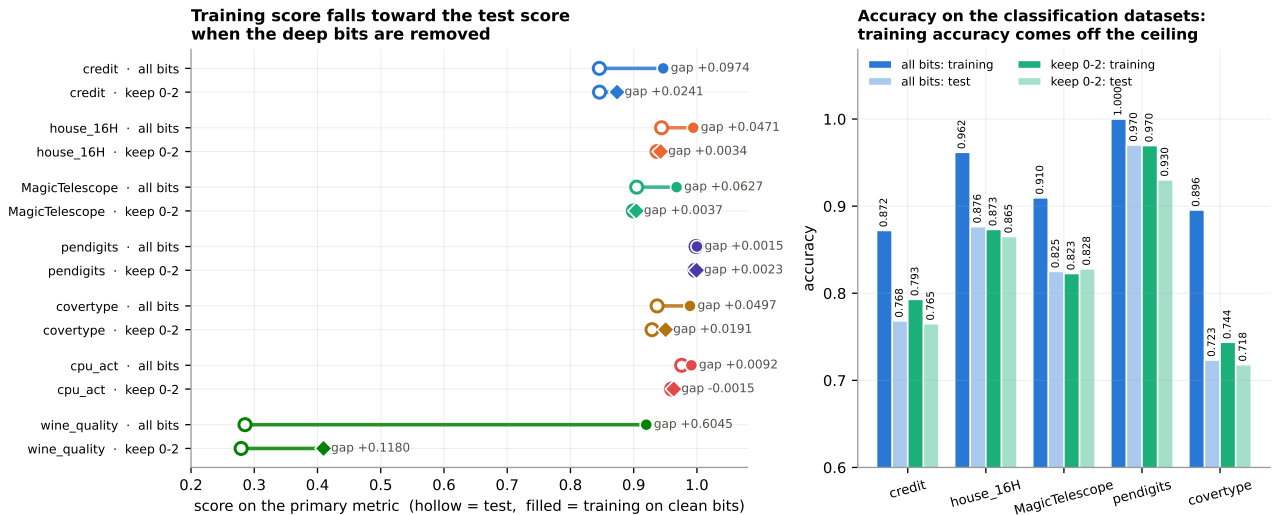


Figure 2: Left: for every dataset, the training score (filled) and test score (hollow) for the full encoding and for depths 0 to 2 only. The line between them is the gap; it shortens on every dataset. Right: training and test accuracy on the classification datasets. Training accuracy comes down from its ceiling while test accuracy barely moves, except on the ten-class problem.

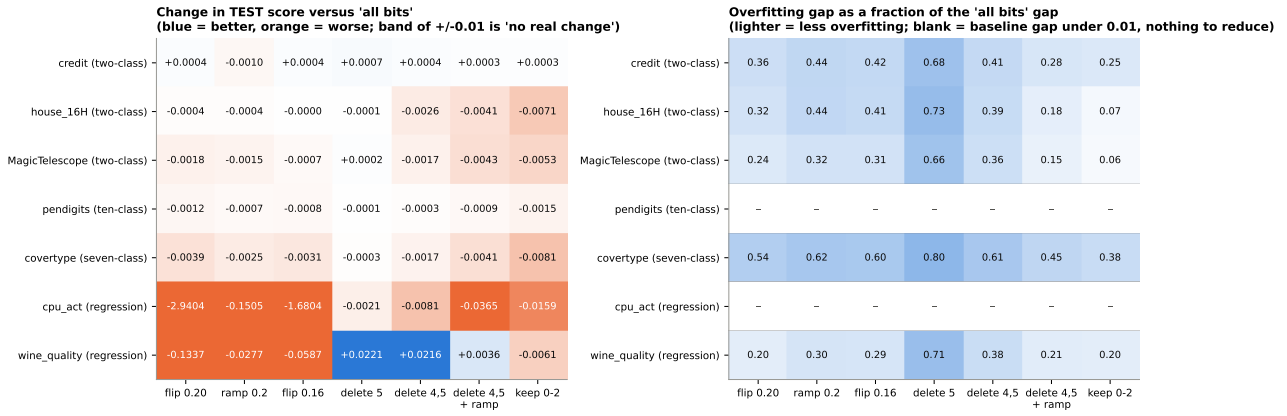


Figure 3: Every arm on every dataset. Left: change in test score against the baseline; values inside ± 0.01 are not real differences. The two flat-noise arms on `cpu_act` are off the scale, at -2.94 and -1.68 . Right: the gap as a fraction of the baseline gap. Blank rows are the two datasets whose baseline gap was under 0.01 , so there was nothing to reduce.

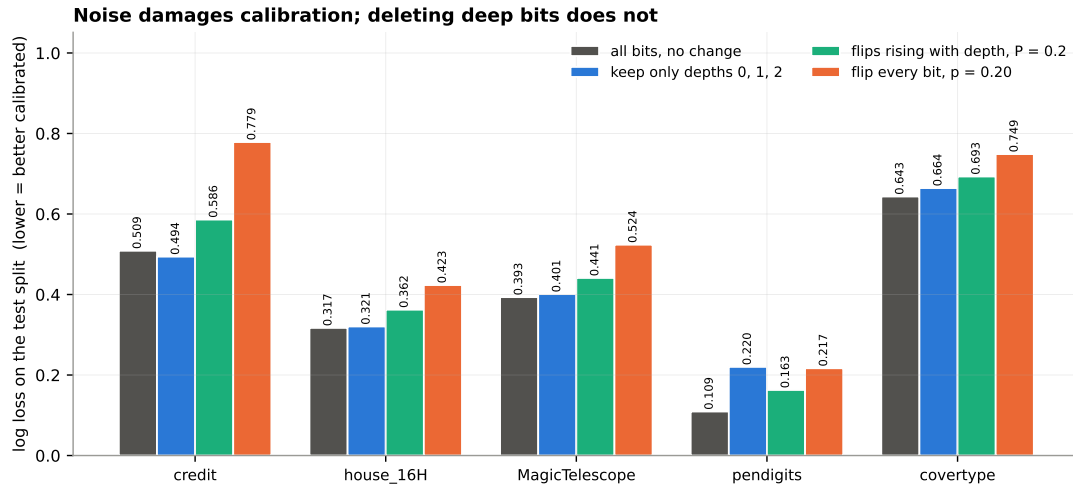


Figure 4: Log loss on the test split for the classification datasets. On the two-class problems and on `covertime`, deleting depths leaves it roughly where it was, while flipping bits raises it by 16 to 53 percent, meaning the model's probabilities become less trustworthy even when its ranking does not change. On the ten-class `pendigits` both deletion and noise double it.

Depth-shaped noise versus flat noise at the same total amount

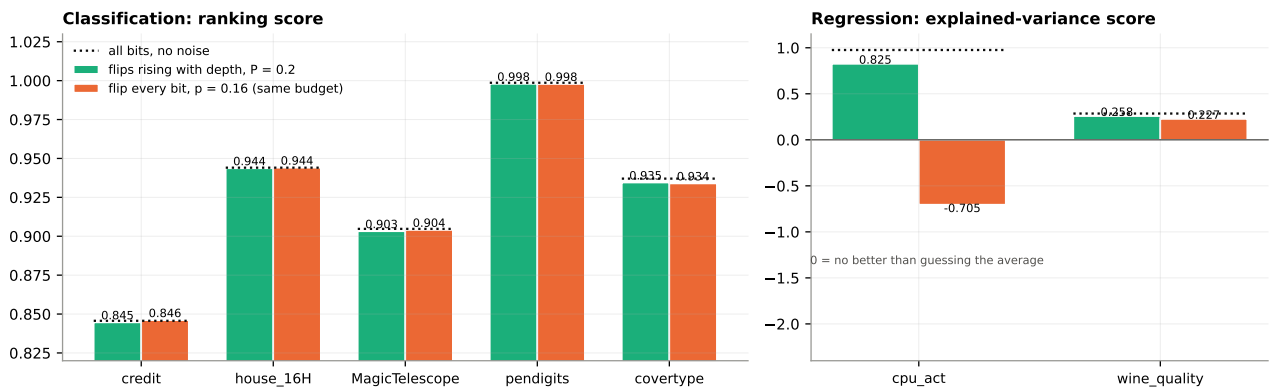


Figure 5: The depth ramp beside the flat control that tells the same total number of lies. On classification they tie. On regression, flat noise is catastrophic, because it also flips the root bits that carry most of the signal, and even the ramp costs a lot.

5.2 Every arm on every dataset

Each table lists all eight arms plus the raw-features floor and the best tree. Training scores are on clean bits. The recommended arm from credit is in bold.

credit (two classes, the anchor dataset)

Arm	Bits	Flip rate	Ranking score			Gap	Accuracy		Log loss test
			train	val	test		train	test	
raw features only (no tree bits)	10	–	0.7466	0.7317	0.7288	+0.0149	0.6674	0.6496	0.6310
all bits, no change	5401	–	0.9466	0.8492	0.8457	+0.0974	0.8721	0.7683	0.5086
flip every bit, $p = 0.20$	5401	0.200	0.8840	0.8489	0.8460	+0.0351	0.7562	0.7338	0.7787
flips rising with depth, $P = 0.2$	5401	0.159	0.8914	0.8483	0.8446	+0.0431	0.7911	0.7588	0.5861
flip every bit, $p = 0.16$ (same budget as the ramp)	5401	0.160	0.8899	0.8490	0.8461	+0.0409	0.7722	0.7429	0.6733
delete depth 5	2935	–	0.9162	0.8497	0.8464	+0.0665	0.8355	0.7675	0.4993
delete depths 4 and 5	1480	–	0.8899	0.8497	0.8460	+0.0402	0.8096	0.7675	0.4955
delete 4 and 5, then ramp 0.2	1480	0.025	0.8763	0.8491	0.8460	+0.0271	0.7857	0.7650	0.5142
keep only depths 0, 1, 2	700	–	0.8737	0.8496	0.8459	+0.0241	0.7930	0.7650	0.4941
best tree model (lightgbm)	0.8453								

house_16H (two classes)

Arm	Bits	Flip rate	Ranking score			Gap	Accuracy		Log loss test
			train	val	test		train	test	
raw features only (no tree bits)	16	–	0.8856	0.8869	0.8850	-0.0012	0.8156	0.8182	0.4578
all bits, no change	5756	–	0.9941	0.9471	0.9441	+0.0471	0.9619	0.8765	0.3170
flip every bit, $p = 0.20$	5756	0.200	0.9611	0.9462	0.9437	+0.0149	0.8811	0.8651	0.4234
flips rising with depth, $P = 0.2$	5756	0.161	0.9671	0.9465	0.9437	+0.0206	0.8941	0.8671	0.3623
flip every bit, $p = 0.16$ (same budget as the ramp)	5756	0.160	0.9659	0.9467	0.9440	+0.0192	0.8925	0.8681	0.3835
delete depth 5	3032	–	0.9808	0.9463	0.9439	+0.0345	0.9323	0.8755	0.3082
delete depths 4 and 5	1497	–	0.9625	0.9440	0.9415	+0.0184	0.8995	0.8701	0.3090
delete 4 and 5, then ramp 0.2	1497	0.024	0.9517	0.9431	0.9400	+0.0086	0.8816	0.8715	0.3203
keep only depths 0, 1, 2	700	–	0.9425	0.9392	0.9369	+0.0034	0.8733	0.8651	0.3205
best tree model (lightgbm)	0.9526								

MagicTelescope (two classes)

Arm	Bits	Flip rate	Ranking score			Gap	Accuracy		Log loss test
			train	val	test		train	test	
raw features only (no tree bits)	10	–	0.8418	0.8376	0.8486	+0.0042	0.7724	0.7698	0.4835
all bits, no change	4684	–	0.9678	0.9051	0.9047	+0.0627	0.9096	0.8251	0.3935
flip every bit, $p = 0.20$	4684	0.200	0.9184	0.9033	0.9029	+0.0151	0.8343	0.8186	0.5237
flips rising with depth, $P = 0.2$	4684	0.155	0.9247	0.9049	0.9032	+0.0198	0.8470	0.8216	0.4411
flip every bit, $p = 0.16$ (same budget as the ramp)	4684	0.160	0.9240	0.9045	0.9040	+0.0196	0.8455	0.8246	0.4747
delete depth 5	2752	–	0.9470	0.9054	0.9050	+0.0417	0.8787	0.8296	0.3892
delete depths 4 and 5	1454	–	0.9257	0.9033	0.9030	+0.0224	0.8514	0.8271	0.3925
delete 4 and 5, then ramp 0.2	1454	0.028	0.9110	0.9014	0.9004	+0.0096	0.8323	0.8236	0.4042
keep only depths 0, 1, 2	698	–	0.9041	0.9005	0.8994	+0.0037	0.8227	0.8281	0.4010
best tree model (xgboost)	0.9377								

pendigits (ten classes)

Arm	Bits	Flip rate	Ranking score			Gap	Accuracy		Log loss test
			train	val	test		train	test	
raw features only (no tree bits)	16	–	0.9911	0.9911	0.9906	-0.0001	0.8980	0.9005	0.4524
all bits, no change	5517	–	1.0000	0.9985	0.9986	+0.0015	1.0000	0.9703	0.1093
flip every bit, p = 0.20	5517	0.200	0.9996	0.9973	0.9974	+0.0023	0.9645	0.9375	0.2168
flips rising with depth, P = 0.2	5517	0.159	0.9999	0.9978	0.9979	+0.0021	0.9818	0.9478	0.1628
flip every bit, p = 0.16 (same budget as the ramp)	5517	0.160	0.9999	0.9977	0.9978	+0.0021	0.9783	0.9460	0.1745
delete depth 5	3024	–	1.0000	0.9984	0.9985	+0.0016	1.0000	0.9648	0.1211
delete depths 4 and 5	1498	–	1.0000	0.9980	0.9983	+0.0020	0.9960	0.9557	0.1493
delete 4 and 5, then ramp 0.2	1498	0.025	0.9997	0.9974	0.9977	+0.0023	0.9727	0.9466	0.1715
keep only depths 0, 1, 2	700	–	0.9993	0.9969	0.9971	+0.0023	0.9696	0.9303	0.2202
best tree model (lightgbm)	0.9995								

covertypes (seven classes)

Arm	Bits	Flip rate	Ranking score			Gap	Accuracy		Log loss test
			train	val	test		train	test	
raw features only (no tree bits)	54	–	0.9238	0.9188	0.9106	+0.0050	0.6970	0.6979	0.7367
all bits, no change	4339	–	0.9890	0.9394	0.9370	+0.0497	0.8956	0.7233	0.6434
flip every bit, p = 0.20	4339	0.200	0.9619	0.9352	0.9332	+0.0267	0.7696	0.7163	0.7488
flips rising with depth, P = 0.2	4339	0.154	0.9675	0.9369	0.9345	+0.0306	0.7871	0.7196	0.6931
flip every bit, p = 0.16 (same budget as the ramp)	4339	0.160	0.9662	0.9364	0.9339	+0.0298	0.7804	0.7179	0.7116
delete depth 5	2554	–	0.9797	0.9399	0.9368	+0.0398	0.8435	0.7233	0.6331
delete depths 4 and 5	1394	–	0.9670	0.9367	0.9354	+0.0303	0.7925	0.7238	0.6350
delete 4 and 5, then ramp 0.2	1394	0.029	0.9561	0.9337	0.9329	+0.0223	0.7578	0.7179	0.6512
keep only depths 0, 1, 2	689	–	0.9505	0.9314	0.9290	+0.0191	0.7438	0.7179	0.6644
best tree model (xgboost)	0.9654								

cpu_act (regression; typical error and average absolute error in percentage points of processor time)

Arm	Bits	Flip rate	Explained variance			Gap	Typical error		Avg. abs. error test
			train	val	test		train	test	
raw features only (no tree bits)	21	–	0.7247	0.7070	0.7638	+0.0177	9.540	8.974	5.695
all bits, no change	3507	–	0.9912	0.9820	0.9758	+0.0092	1.707	2.874	1.956
flip every bit, p = 0.20	3507	0.200	-2.0525	-1.6278	-1.9647	-0.4247	31.765	31.794	30.880
flips rising with depth, P = 0.2	3507	0.153	0.8285	0.8456	0.8253	-0.0171	7.529	7.718	6.828
flip every bit, p = 0.16 (same budget as the ramp)	3507	0.160	-0.7566	-0.5114	-0.7046	-0.2453	24.097	24.109	23.398
delete depth 5	1939	–	0.9849	0.9805	0.9737	+0.0043	2.236	2.997	2.039
delete depths 4 and 5	1116	–	0.9770	0.9755	0.9677	+0.0015	2.757	3.318	2.296
delete 4 and 5, then ramp 0.2	1116	0.026	0.9433	0.9472	0.9393	-0.0040	4.331	4.549	3.461
keep only depths 0, 1, 2	629	–	0.9634	0.9648	0.9599	-0.0015	3.480	3.700	2.632
best tree model (lightgbm)	0.9867								

wine_quality (regression; errors in quality-score points)

Arm	Bits	Flip rate	Explained variance			Gap	Typical error		Avg. abs. error test
			train	val	test		train	test	
raw features only (no tree bits)	11	–	0.2936	0.2945	0.2800	-0.0008	0.731	0.748	0.586
all bits, no change	5498	–	0.9199	0.3153	0.2854	+0.6045	0.246	0.745	0.585
flip every bit, p = 0.20	5498	0.200	0.3219	0.2030	0.1518	+0.1190	0.716	0.812	0.640
flips rising with depth, P = 0.2	5498	0.159	0.4797	0.2992	0.2577	+0.1805	0.628	0.760	0.595
flip every bit, p = 0.16 (same budget as the ramp)	5498	0.160	0.4374	0.2642	0.2267	+0.1732	0.653	0.775	0.607
delete depth 5	2993	–	0.7358	0.3055	0.3075	+0.4303	0.447	0.734	0.575
delete depths 4 and 5	1493	–	0.5443	0.3169	0.3071	+0.2274	0.587	0.734	0.570
delete 4 and 5, then ramp 0.2	1493	0.025	0.4370	0.3116	0.2890	+0.1254	0.653	0.744	0.577
keep only depths 0, 1, 2	700	–	0.4096	0.2916	0.2793	+0.1180	0.668	0.749	0.585
best tree model (random forest)	0.4722								

6 Observations

1. **The observation holds on 5 of 7 datasets, and the two exceptions are exactly the two datasets that barely overfit to begin with.** On pendigits the baseline gap is only 0.0015 (training ranking score 1.0000, test 0.9986); there is nothing to remove, and the shallow arm changes test by -0.0015. On cpu_act the baseline gap is 0.0092 and the shallow arm removes it entirely (gap -0.0015) but at a test cost of -0.0159, outside the tolerance. So the rule fails where the premise fails: where there was no overfitting, feature regularisation had no job to do.
2. **Wherever the model did overfit, deleting the deep layers cut the gap by 62 to 94 percent.** Credit 0.0974 to 0.0241 (75 percent), house_16H 0.0471 to 0.0034 (93 percent), MagicTelescope 0.0627 to 0.0037 (94 percent), covertype 0.0497 to 0.0191 (62 percent), wine_quality 0.6045 to 0.1180 (80 percent). The test score moved by at most -0.0081 on those five.
3. **The training score came down from its ceiling on every dataset.** On the primary metric: credit 0.9466 to 0.8737, house_16H 0.9941 to 0.9425, MagicTelescope 0.9678 to 0.9041, pendigits 1.0000 to 0.9993, covertype 0.9890 to 0.9505, cpu_act 0.9912 to 0.9634, wine_quality 0.9199 to 0.4096. Training *accuracy*, the number asked about directly: pendigits was literally 1.0000 and came down to 0.9696; house_16H 0.9619 to 0.8733; covertype 0.8956 to 0.7438; credit 0.8721 to 0.7930; MagicTelescope 0.9096 to 0.8227. The model with 700 bits no longer memorises the training set.
4. **Credit is the only dataset where the linear model matches the best tree.** Measured as the share of the distance from “raw features only” to “best tree” that the full encoding covers: credit 100 percent (0.8457 against a tree of 0.8453), cpu_act 95 percent, pendigits 91 percent, house_16H 87 percent, MagicTelescope 63 percent, covertype 48 percent, wine_quality 3 percent. The bits carry a large part of what the trees know, but on six of seven datasets a linear read-out of them does not recover all of it. The earlier “matches the tree” headline was a property of credit.
5. **Wine quality is the clearest case of memorisation without learning.** With all bits the training explained-variance score is 0.9199 and the test score 0.2854, a gap of 0.6045. The raw features alone reach 0.2800, so the 5498 bits added almost nothing a linear model could use, while giving it enough capacity to fit the training set nearly perfectly. The best tree reaches 0.4722; whatever it knows about wine is not linear in these bits. Here deletion even *improves* test: deleting depth 5 gives 0.3075, the best arm on this dataset.
6. **Deleting regularises cleanly; flipping damages calibration.** Log loss on the test split, the penalty for confident wrong answers, tells the two apart. Baseline, shallow, and flip-every-bit-at-0.20: credit 0.5086, 0.4941, 0.7787; house_16H 0.3170, 0.3205, 0.4234; MagicTelescope 0.3935, 0.4010, 0.5237; covertype 0.6434, 0.6644, 0.7488. On these four, deletion leaves the probabilities about as trustworthy as before while flipping makes them 16 to 53 percent worse, even though the ranking score is unchanged. Pendigits is the exception: baseline 0.1093, shallow 0.2202, flipped 0.2168; there both deletion and noise double the log loss, consistent with the many-class point below. The reason is a mismatch: the model is fitted on bits that are on more often than the clean bits it is scored on, so its scores shift. Ranking ignores a shift; log loss does not.
7. **The same mismatch is catastrophic for regression.** Flat noise at $p = 0.16$ drives cpu_act from 0.9758 to -0.7046, worse than guessing the average, and at $p = 0.20$ to -1.9647. On wine_quality flat noise at 0.16 gives 0.2267 against a baseline of 0.2854. A shifted score that ranking tolerates is a wrong number that a number-prediction score punishes in full.
8. **The depth shape of the noise does not matter for classification and matters enormously for regression.** Ramp against its equal-budget flat control on the five classification datasets: credit 0.8446 against 0.8461, house_16H 0.9437 against 0.9440, MagicTelescope 0.9032 against 0.9040, pendigits 0.9979 against 0.9978, covertype 0.9345 against 0.9339, every pair within

0.0015. On regression the ramp is far better, `cpu_act` 0.8253 against -0.7046 and `wine_quality` 0.2577 against 0.2267, because it never flips the root bits that carry most of a regression forest’s signal. But even the ramp costs `cpu_act` 0.15 of explained variance. Noise, of any shape, is the wrong tool for regression; deletion is the right one.

9. **Many-class problems keep more of their signal in the deep bits.** On `pendigits` the shallow arm loses only 0.0015 of ranking score but 0.040 of accuracy (0.9703 to 0.9303), because telling ten digits apart needs the fine splits that deep bits provide. `Covertime` shows the same in milder form (0.7233 to 0.7179). Ranking score alone would have hidden this; tracking every metric on every split exposed it.
10. **How much to delete is a dial, and the dial reads the same on every dataset.** Deleting only depth 5 never costs more than 0.0021 of test score on any dataset and cuts the gap by 20 to 34 percent. Deleting depths 4 and 5 costs at most 0.0081 and cuts the gap by 39 to 64 percent. Keeping only depths 0 to 2 costs at most 0.0159 and cuts it by 62 to 94 percent. More deletion buys more regularisation at a slowly rising price, with the price highest on regression and many-class problems.
11. **Every number here is from one random split and one seed.** The noise arms were shown earlier to vary by only 0.0002 between fresh noise draws, but the split itself is a larger source of variation, roughly 0.01 on a test set of two thousand rows. Differences smaller than that between arms should be read as ties.

7 Key findings

1. **Feature regularisation works as a regulariser across every kind of problem tested.** On all five datasets where the full encoding overfit, deleting the deep tree layers cut the train-minus-validation gap by 62 to 94 percent and brought the training score off its ceiling, at a test cost of at most 0.008 for deleting two layers and 0.016 for keeping only three. The credit observation was not a fluke of credit.
2. **The “linear model matches the tree” result was specific to credit.** On the other six datasets the linear model covers between 48 and 95 percent of the way from raw features to the best tree, and on wine quality almost none. The tree models remain ahead on six of seven datasets. The encoding carries most of what the trees know; a linear read-out does not recover all of it. This qualifies the earlier report and should be stated plainly in anything written up.
3. **Delete, do not flip.** Deletion has far smaller side effects than noise. On two-class problems it leaves calibration and test scores intact; on regression and many-class problems it costs a little, never more than 0.016 of test score. Bit-flip noise raises log loss by 16 to 98 percent on classification, is destructive on regression, and never beats deletion on the test score. The recommended default is to delete depths 4 and 5, which shrinks the encoding three to four times, cuts overfitting by 39 to 64 percent, and costs at most 0.008 anywhere. On two-class problems, keeping only depths 0 to 2 is free and cuts overfitting by 75 to 94 percent.
4. **Depth-shaped noise is not worth pursuing.** Against an equal-budget flat control it ties on every classification dataset. It wins on regression only because flat noise is catastrophic there, and even then it loses badly to simple deletion. The intuition behind it, that deep splits are the unreliable ones, is correct, and the way to act on it is to delete them.
5. **Track more than one score.** The ranking score alone would have said the shallow encoding is free on `pendigits`; accuracy shows it costs four points. The ranking score alone would have said noise is harmless on classification; log loss shows it damages the probabilities. The request to track all metrics on all splits changed the conclusions, not just the tables.

6. **What this establishes, and what comes next.** With the feature side now characterised across task types, the next step named in the earlier discussion is to put richer heads on the shallow encoding and compare them against this logistic one under the identical protocol. The tree models' lead on six of seven datasets says a nonlinear head has real room to gain; the credit result says it should be judged against a linear baseline that is already strong.

Reproduction. One command runs everything and writes every table, figure and per-run log:

```
python run_feature_reg_suite.py --root results/featreg_suite --bundle
```

Repository: github.com/sushanedulloo/TKCE. Driver notebook: `colab_feature_reg_suite.ipynb`. All numbers in this document, including those in the text, are generated from the run's own result files; none was typed by hand.